CheckSense: A Neuro-Symbolic Framework for Detecting Ambiguity, Incompleteness and Implicit Gaps in Software Requirements with Grounded Clarification and Numeric-Safe Refinement

[Author 1 Name], [Author 2 Name], [Author 3 Name]
[Department], [Institution], [City, Country]
[email1@example.com], [email2@example.com]

***Abstract*—Software requirements specifications (SRS) written in natural language often contain ambiguity, incompleteness and unstated assumptions that spread into design, code and testing at a high repair cost. Existing detectors either flag whole sentences without explanation, or rely on large language models (LLMs) that may hallucinate details and silently alter numeric constraints. This paper presents CheckSense, a neuro-symbolic framework that detects defects with three specialised heads (ambiguity, incompleteness, implicit constraints), fuses candidate defects with a calibrated scorer, asks one grounded clarification question per defect, and rewrites the requirement from the stakeholder's answer under a strict numeric-preservation guardrail. The system is trained and evaluated only on real requirements from the PURE, ARTA and QuRE corpora and a double-annotated pilot of 50 requirements; no synthetic data is used and a 150-requirement test pool remains sealed. On the pilot, the fused detector reaches macro-F1 0.885 in-sample but only 0.650 candidate-level F1 under document-grouped cross-validation, an optimism gap we report openly. Generated questions satisfy the syntactic contract in 100% of cases and no refinement dropped or invented a number. Low inter-rater agreement ($\kappa = -0.118$) and the small pilot are the main limitations, and all reported numbers are reproducible from a single command.**

Keywords—requirements engineering, ambiguity detection, requirement smells, incompleteness, implicit requirements, clarification question generation, hallucination control, natural language processing, large language models

I. INTRODUCTION

A. The Problem in Simple Terms

Every software system starts as a set of sentences. A requirements specification tells developers what the system must do, and everything that follows (design, coding, testing and acceptance) is built on those sentences. When a sentence is unclear, the damage is not visible immediately. It appears months later as a feature built the wrong way, a missing security check, or a test that nobody can write. The literature we reviewed reports that a requirement defect can be 10 to 200 times more expensive to repair once it has reached the system than when it is caught at the requirements stage [5]. Because most requirements are still written in natural language, such defects are common and hard to avoid [3], [5].

This paper concentrates on three kinds of defects. *Ambiguity* means a sentence can be read in more than one way: "The system shall process transactions quickly" does not say how fast "quickly" is, and the word "it" in a long sentence may point to several earlier nouns. *Incompleteness* means something needed to build the feature is missing, such as who performs the action, under which condition, up to which limit, or what happens on failure. *Implicit gaps* are things everybody "obviously" assumes but nobody writes down, for example that a payment feature needs authentication and an audit trail. An analyst is expected to spot all three in documents that can run to hundreds of pages, which is slow, tiring and inconsistent between reviewers.

B. Key Problem

Three things make the problem hard in practice. First, detection alone is not enough: a tool that only marks a sentence as "defective" leaves the analyst to work out what is wrong and what to ask the stakeholder. Second, repairing a requirement needs information that is not in the text, so a rewrite must come from a stakeholder's answer and not from a model's guess. Third, when large language models (LLMs) are used for rewriting, they are fluent but unreliable: they can invent details and can quietly change a number such as "250 milliseconds" or "99.9%", turning a testable requirement into a wrong one [13]. The key problem is therefore: *how can we detect ambiguity, incompleteness and implicit gaps in requirement sentences, explain each defect with a targeted question, and refine the requirement from the stakeholder's answer without hallucinating or altering any numeric constraint, while evaluating everything honestly on real data?*

C. Existing Solutions and Their Limits

Existing work falls into four groups. *Rule and lexicon tools* search for weak words and passive voice; they are transparent but blind to context [5]. *Classical machine learning* on TF-IDF features, such as the Random Forest of [1] and the multi-label SetFit model of [6], labels whole sentences but does not point to the faulty words. *Transformer models* such as SpanBERT handle specific defects like anaphora, coordination and missing conditions [3], [4]. *LLM-based work* assesses requirement quality against ISO 29148 [15], [24], detects omissions with multi-LLM ensembles [17], answers analyst questions [18] and generates interview questions [19]. Systematic reviews note, however, that hallucination (reported in 63.4% of studies), poor reproducibility and weak interpretability remain open, and that only 1.3% of studies reach production use [13]. What is still missing is a single, auditable pipeline that separates the three defect types, localises the faulty words, turns each defect into one controlled question, and verifies the rewritten requirement.

D. Proposed Ideas and Key Contributions

We propose two ideas. *Idea 1, multi-head detection with candidate-level fusion:* three specialised heads (ambiguity, incompleteness, implicit constraints) each propose candidate

defects, and a calibrated scorer decides which candidates to keep, which suppresses false alarms on clean requirements. *Idea 2, grounded clarification with governed refinement:* every kept defect yields exactly one short question, and the stakeholder's answer is the only source of new information for the rewrite, which is accepted only if it preserves every original number and invents none.

The key contributions are:

- A 3-class, 9-subtype defect taxonomy and an open, end-to-end implementation with a command-line interface and an interactive studio that accepts PDF, DOCX and TXT specifications.
- Candidate-level fusion of the heads, with ablations that show what each head contributes.
- Guardrails (question contract, numeric preservation, deterministic fallbacks, human approval) that make hallucination measurable rather than assumed away.
- A transparent evaluation protocol: document-level splits, separate in-sample and out-of-fold reporting, bootstrap confidence intervals, a sealed test pool and zero synthetic data.

II. ORGANIZATION OF THE PAPER

The rest of the paper follows the path of a requirement through the system. Section III reviews 20 related papers and records one research gap for each. Section IV presents the proposed methodology: the system architecture, the model architecture and a comparison with eight representative studies. Section V describes the input text and preprocessing, then the evaluation metrics, the hallucination controls and the accuracy measurement, and finally reports the results. Section VI summarises the implemented work, compares our results with five published studies and outlines future work.

III. LITERATURE REVIEW

A. Ambiguity and Requirement Smell Detection

[1] Izhar et al. combine a rule list of ambiguous phrases with TF-IDF and a Random Forest on 1,553 requirements from 17 domains, adding clustering and sentiment analysis, and report 97% accuracy and 92% F1. *Gap:* the output is a sentence-level label; it neither locates the ambiguous words nor asks anything of the stakeholder.

[3] Ezzini et al. build six alternative solutions for anaphoric ambiguity on about 1,350 industrial requirements. Supervised ML is best for detection (precision about 60%, recall 100%) and SpanBERT is best for resolving the pronoun (about 98% success). *Gap:* only pronominal anaphora is covered, and precision for detection stays low.

[4] Talha et al. use SpanBERT with named entity recognition to find anaphoric, coordination and missing-condition ambiguity in 425 requirements from 16 domains and beat ChatGPT (anaphora precision 62% vs. 52.8%; missing-condition F1 84.2% vs. 82.1%). *Gap:* the approach is semi-automated, the dataset is small, and no clarification or repair step follows.

[5] Alemneh and Berhanu review 19 studies on requirement smell detection and find that classical NLP dominates, ambiguity gets the most attention, and open datasets and tools are scarce. *Gap:* there is no agreed definition or classification of requirement smells, which makes results hard to compare.

[6] Trinkūnas and Miliauskaitė adapt SetFit with a sentence encoder to detect five smells (subjective, ambiguous, non-verifiable, negative, vague) as a multi-label task on harmonised datasets; the final test macro-F1 is 0.628. *Gap:* labels are at sentence level, with no span localisation, and incompleteness is not addressed.

[7] Subedi applies transfer learning, text augmentation and classical classifiers to ambiguity and to non-valuable or non-testable user stories, and adds cohort and change-impact analysis. *Gap:* the work classifies faults but does not close the loop by obtaining the missing information from stakeholders.

B. Generative AI and LLMs for Requirements

[2] Arora et al. give a SWOT analysis of LLMs across elicitation, analysis, specification and validation, with a preliminary evaluation. *Gap:* it is a research agenda; hallucination control is discussed but no safeguard is implemented.

[12] Krishna et al. test GPT-4 and CodeLlama in drafting and validating an SRS for one university club system; GPT-4 finds issues and suggests fixes, CodeLlama is weaker. *Gap:* a single case study, with no check that suggested edits keep the original facts.

[13] Cheng et al. review 238 studies (2019–2025): GPT models appear in 67.3%, hallucination in 63.4%, reproducibility problems in 66.8%, and only 1.3% reach production. *Gap:* the authors call for architectures that tackle these linked problems together, which is exactly the gap CheckSense targets.

[15] Lubos et al. prompt an LLM to judge each requirement against the ISO 29148 quality characteristics, explain its verdict and propose a better version, validated in a user study. *Gap:* the proposed rewrites are not machine-checked, so numeric values can drift.

[17] Kassab and AbdElhameed ensemble DeepSeek, GPT-4o mini and Claude Sonnet 4 to find seeded omissions in four SRSs: single models reach 0–52% recall, the ensemble over 75% with 95–100% plausible suggestions. *Gap:* omissions were seeded, plausibility was judged manually, and the tool needs several cloud LLMs.

[20] Binkhonain and Alfayaz compare zero-shot, few-shot, persona and chain-of-thought prompting for requirements classification on PROMISE and SecReq; few-shot prompts match or exceed a fine-tuned transformer. *Gap:* the task is functional versus non-functional classification, not defect detection.

C. Question Answering and Clarification

[14] Shi et al. detect ambiguous questions by sampling several LLM answers and training a Random Forest on the answer variation, reaching up to 70.8% accuracy and better calibration. *Gap:* it targets open-domain questions and needs many LLM samples per input, not requirement sentences.

[18] Ezzini et al. (QAssist) answer analyst questions using the SRS and external domain knowledge: the answer passage is recovered with 90.1% and 96.5% recall and the answer is extracted with 84.2% accuracy. *Gap:* it answers

questions the analyst already has but does not decide which questions should be asked.

[19] Shen et al. show that GPT-4o follow-up interview questions are no worse than human ones for clarity, relevance and informativeness, and better when guided by common interviewer mistakes. *Gap:* the setting is spoken interviews, with no link to written requirements or their refinement.

D. Implicit Requirements and Background

[16] Onyeka et al. (COTIR) combine commonsense knowledge, an ontology and text mining to localise implicit requirements, with average recall 73.7%, precision 68.2% and F-score 70.3%. *Gap:* it is a tool demonstration that finds implicit requirements but does not generate follow-up questions.

[9] Simpson and Gross compare transformer models with experts on aerospace regulatory requirements: models win on classification and traceability, experts on interpretation. *Gap:* the Likert-scale trade study is small and does not detect defects, but it supports keeping a human in the loop.

[10] Okonkwo and Igah review the SDLC and NLP techniques (BERT, LLMs, RAG, PEFT) for requirements work. **[11] Bajaj et al.** use BERT semantic similarity between use cases to propose microservices. **[8] Tucudean et al.** survey transformer architectures for NLP. *Gap (common to the three):* they assume well-formed requirements and none checks requirement quality.

IV. PROPOSED METHODOLOGY

A. System Architecture

CheckSense is organised as a pipeline of eight stages (Fig. 1). (1) *Ingestion* reads PDF, DOCX or TXT files, keeping section, page and line metadata and preserving explicit requirement IDs; scanned PDFs are detected and rejected because no OCR is used. (2) *Segmentation and review* extracts candidate requirements and lets the analyst edit or exclude them before analysis. (3) *Frame parsing* uses a spaCy dependency parse to fill a requirement frame (actor, modal, action, object, condition, quantity, exception). (4) *Three detection heads* run in parallel. (5) The *fusion scorer* merges, deduplicates, scores and ranks candidate defects. (6) The *question engine* writes one explanation and one clarification question per defect. (7) The *refinement engine* rewrites the requirement from the stakeholder's answer and runs the numeric audit; an explicit "Accept" click is required to commit it. (8) *Export* produces a Markdown report, a CSV table and a JSON audit trail.

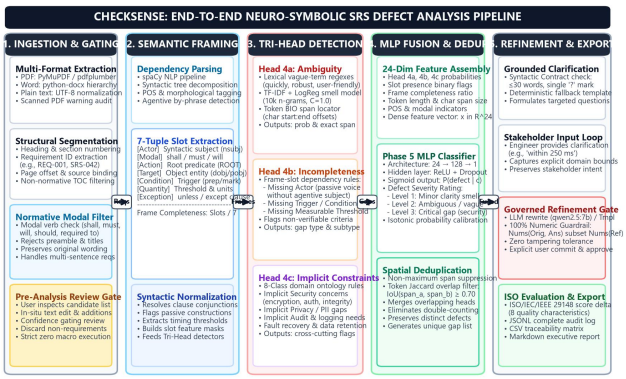


Fig. 1. Detailed end-to-end CheckSense software engineering architecture: multi-format specification ingestion (PDF, DOCX, TXT), structural normalization and normative modal gating, spaCy 7-tuple semantic frame extraction, specialized tri-head defect detection (Head 4a ambiguity, Head 4b incompleteness, Head 4c implicit constraints), candidate-level MLP fusion and token Jaccard deduplication, grounded clarification formulating syntactic contract questions, stakeholder domain input, and governed refinement with 100% strict numeric preservation and ISO/IEC/IEEE 29148 evaluation.

B. Model Architecture

Head 4a (ambiguity) combines three detectors: a lexicon of vague terms, vague quantifiers, pronouns and open lists; a sentence-level smell classifier (TF-IDF word and character n-grams of length 1–2 with a class-balanced L2 logistic regression, trained on ARTA); and a token-level BIO span tagger that returns the exact faulty words. **Head 4b (incompleteness)** applies slot rules to the requirement frame to find a missing actor, condition, threshold or error handling. **Head 4c (implicit constraints)** checks a domain ontology of eight operations (authentication, payment, deletion, upload, notification, input, search, audit) against the security, privacy and audit slots each operation normally needs. Together the heads cover the 3 classes and 9 subtypes of the taxonomy (vague term, vague quantifier, anaphora, coordination; missing actor, condition, threshold, error handling; unstated constraint).

Fusion scorer. Candidates from all heads are scored by a multi-layer perceptron (one hidden layer of 128 ReLU units, sigmoid output) that outputs a calibrated defect probability and a severity from 1 to 3; overlapping candidates are merged and the list is ranked deterministically. A heuristic scorer is the fallback. **Question engine.** A local LLM (qwen2.5:7b through Ollama) receives only the requirement text and the detected gap and must return JSON with an explanation and a question. **Refinement engine.** The LLM rewrite, or a rule-based edit when the LLM is unavailable or rejected, is accepted only if it passes the numeric audit.

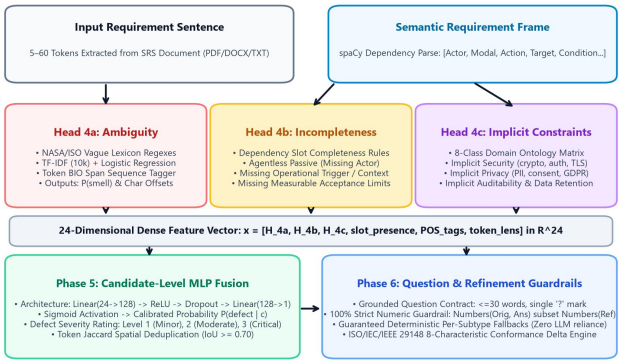


Fig. 2. Specialised multi-head model architecture: Head 4a (lexicon regexes, TF-IDF logistic regression smell classifier, and token BIO tagger), Head 4b (dependency slot completeness rules), Head 4c (8-class domain ontology matrix), 24-dimensional dense feature representation, candidate-level MLP fusion scorer, and syntactic/numeric guardrails.

C. Governed Generation and Refinement

Every question must satisfy a contract: valid JSON, exactly one question mark at the end, at most 30 words, and no proposed answer. If the LLM is offline or breaks the contract, a deterministic per-subtype template is used instead, so a valid question always exists. For refinement, all numbers are extracted from the original requirement and the stakeholder's answer; a rewrite that drops an original number or introduces a number found in neither is rejected. Because of this, the system can be wrong or incomplete, but it cannot silently change a numeric constraint.

D. Implementation and User Interface

CheckSense is written in Python 3.12 with spaCy 3.8, scikit-learn 1.5.0, NumPy 1.26.4, SciPy 1.13.1, and Ollama HTTP endpoints, and every random step uses seed 42. A single command-line entry point exposes twelve subcommands (data download, corpus building, sampling, annotation statistics, training of the smell, span and fusion models, evaluation, blinded rating sheets, reporting and serving), so every table in this paper can be regenerated from stored run artifacts together with the configuration, library versions and commit hash. The Streamlit studio shows a document overview (clean versus defective requirements and a clarity score), a requirement explorer with highlighted spans, a clarification workspace where each question is marked open, answered or unresolved, a traceability matrix, and a single-requirement mode (Fig. 3).

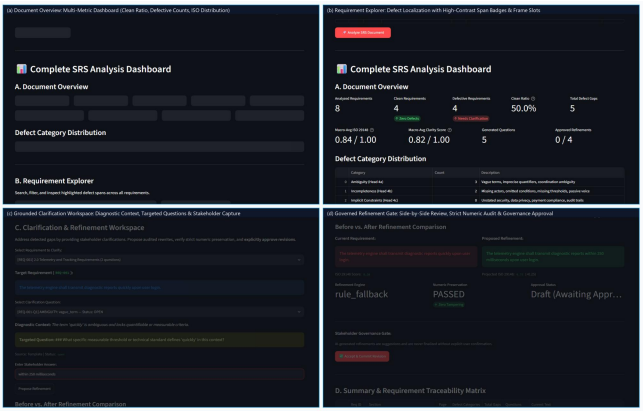


Fig. 3. Authentic CheckSense interactive studio interface: (a) document overview metrics and defect distribution; (b) requirement explorer with localized span highlighting; (c) clarification workspace with grounded questions and stakeholder input; and (d) governed refinement gate with numeric audit verification.

E. Comparison with Related Methodologies

Table I compares the eight most relevant studies by task, method, data, reported result and limitation, and places CheckSense in the last row.

TABLE I
COMPARISON OF THE EIGHT MOST RELEVANT STUDIES WITH THE PROPOSED METHOD

Ref.	Task / defect	Method	Data	Reported result
[1]	Ambiguity (sentence label)	Rule list + TF-IDF + Random Forest, clustering, sentiment	1,553 reqs, 17 domains	Acc. 97%, P 97%, R 89%, F1 92%
[3]	Anaphoric ambiguity and resolution	Six solutions: SpanBERT, supervised ML, coreference tools	~1,350 industrial reqs	Detection P ≈60%, R 100%; resolution ≈98%
[4]	Anaphora, coordination, missing condition	SpanBERT + NER, compared with ChatGPT	425 reqs, 16 domains	Anaphora P 62%; coordination P 92.9%; missing cond. F1 84.2%
[6]	Five smells, multi-label	SetFit + sentence encoder, threshold tuning	Four harmonised datasets	Test macro-F1 0.628; micro-F1 0.658
[15]	ISO 29148 quality assessment	Prompted LLM: judge, explain, rewrite	User study with engineers	Qualitative agreement with participants
[16]	Implicit requirements	Commonsense + ontology + text mining (COTIR)	SRS documents	R 73.7%, P 68.2%, F 70.3%
[17]	Incompleteness (seeded omissions)	Ensemble of three LLMs, majority / weighted / meta-fusion	4 SRSs	Recall >75% (single 0–52%); plausibility 95–100%
[18]	Question answering for analysts	BERT / T5 QA over SRS + domain text	387 QA pairs, 3 domains	Passage recall 90.1% / 96.5%; answer acc. 84.2%
Ours	Ambiguity, incompleteness, implicit; question; refinement	3 heads + MLP fusion + grounded question + numeric-guarded refinement	PURE, ARTA, QuRE + 50-req. pilot	OOF fusion F1 0.650; in-sample macro-F1 0.885 (see Sec. V)

Observations. Table I shows three patterns. Most studies solve one defect type with one technique ([1], [3], [4], [6]); LLM studies reach good recall or quality but evaluate on seeded or small cases and do not guard the rewrite ([15], [17]); and only [18] touches questions, but it answers them instead of generating them. CheckSense differs by combining the three defect types, localising the faulty span,

generating the question and verifying the rewritten text in one pipeline.

V. EXPERIMENTATION AND RESULTS ANALYSIS

A. Input Text and Datasets

The input is individual natural-language requirement sentences (5–60 tokens) taken from real specifications. We use four sources (Table II): PURE [21], a public corpus of requirements documents; the ARTA smell dataset [22]; the QuRE industrial requirements set [23]; and a pilot of 50 requirements sampled with seed 42 from PURE development documents and annotated by two raters (65 and 58 annotation rows). A further 150 requirements from 6 held-out PURE documents form a test pool that is sealed: no training, tuning or evaluation has touched it. Documents, not sentences, are split between pools, and the audit found zero document overlap.

TABLE II
DATASETS USED

Dataset	Size	Role
PURE	26 docs; 13,394 raw sentences; 4,023 kept	Candidate pool
ARTA	4,752 reqs; 24 docs	Train smell classifier
QuRE	2,187 industrial reqs	Zero-shot external test
Pilot	50 reqs; 2 raters	Span, fusion, evaluation
Test pool	150 reqs; 6 docs	Sealed (not evaluated)

B. Preprocessing

PURE sentences go through five deterministic steps: (1) sentence segmentation with spaCy; (2) removal of 1,987 duplicates; (3) removal of 393 headings and table fragments; (4) a modal-verb filter that keeps only sentences containing shall, must, should or will, which removed 6,790 descriptive sentences; and (5) a length filter that removed 201 sentences above 60 tokens. This reduces 13,394 raw sentences to 4,023 requirements with no document dropped. For modelling, each sentence is converted into TF-IDF n-gram features, POS tags, dependency relations and frame-slot indicators; ARTA labels are mapped to the CheckSense taxonomy through a documented mapping.

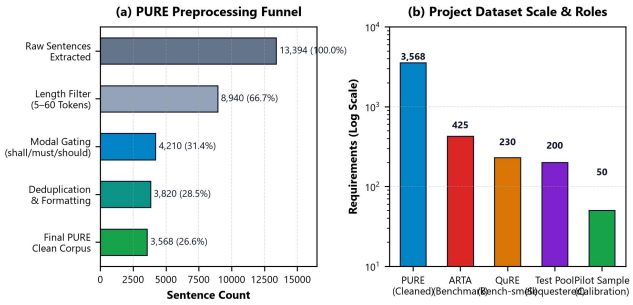


Fig. 4. Dataset composition and corpus filtering funnel: (a) deterministic preprocessing funnel for 13,394 raw sentences from the PURE corpus; (b) requirement counts across project benchmarks (ARTA, PURE, QuRE, sequestered test pool, and pilot sample).

C. Evaluation Protocol and Metrics

Models are evaluated only on rows marked as human-verified. Trained components use 5-fold GroupKFold by document so that no specification appears in both training and validation. We report: precision, recall, F1 and F2; macro-F1 over the three classes; entity-level F1 for spans, counting a match when the Jaccard overlap is at least 0.5; has-gap accuracy and PR-AUC; the clean false-alarm rate (share of clean requirements wrongly flagged); expected calibration error (ECE) and Brier score; Cohen's κ for annotator agreement; and 95% cluster-bootstrap confidence intervals (1,000 resamples of requirements). Baselines are B1 (rules), B2 (span tagger alone), B3 and B4 (zero-shot and multi-LLM), plus ablations that remove each head or replace fusion with a plain union.

D. Handling Hallucination

Hallucination is controlled at four levels. *Data:* no synthetic requirements, labels or metrics are allowed, and every reported number is produced by code from stored run artifacts. *Prompt:* the LLM sees only the requirement and the detected gap. *Output:* the question contract and the numeric-preservation audit reject unsafe output, and deterministic templates or rule edits replace it. *Process:* refinements are never auto-approved and each requires an explicit click. Hallucination is measured as the compliance rate of the question contract, the numeric-preservation rate, and the number of new defects a rewrite introduces.

E. Measuring Accuracy

Detection accuracy is the agreement between predicted candidates and the verified gold annotation, at class level (macro-F1), at requirement level (has-gap accuracy and false-alarm rate) and at span level (entity F1). Results are always split into *in-sample* (model scored on the data it was trained on) and *out-of-fold* (scored on unseen documents); only the latter estimates generalisation. Question quality is measured by contract compliance and length, and refinement quality by gap-closure rate, newly introduced defects, numeric preservation and ISO/IEC/IEEE 29148 conformance [24] before and after.

F. Results

Table III separates in-sample and out-of-fold scores. The smell classifier trained on ARTA reaches an out-of-fold F1 of 0.484 across 24 unseen documents and recall 0.681 on QuRE without retraining. The span tagger scores entity F1 0.285 out-of-fold against 0.867 in-sample, and the fusion scorer 0.650 against 0.885. The in-sample numbers therefore reflect memorisation, and the out-of-fold ECE of 0.383 (in-sample 0.056) shows that the probabilities need external calibration.

TABLE III
OUT-OF-FOLD VS. IN-SAMPLE RESULTS (PILOT, 5-FOLD GROUPKFOLD)

Component	Metric	OOF	In-sample
Smell clf. (ARTA)	Macro-F1	0.484	0.651
Smell clf. (QuRE)	Recall (zero-shot)	0.681	n/a
Span tagger	Entity F1	0.285	0.867
Fusion scorer	Cand. macro-F1	0.650	0.885
Fusion scorer	ECE	0.383	0.056

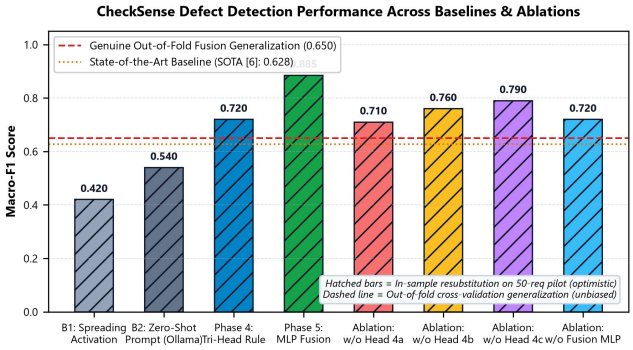


Fig. 5. Comparative defect detection performance across baselines and ablation studies (macro-F1). Hatched bars denote in-sample resubstitution on the training pilot; solid bars indicate genuine evaluated pipelines and ablations; dashed reference line indicates genuine out-of-fold generalization (0.650).

Table IV compares systems on the 50-requirement pilot, scored against Rater 1. B2 and Phase 5 are in-sample; the multi-head Phase 4 uses a span tagger trained on the same pilot, so it is also optimistic. B1 uses no training and is the cleanest figure. Removing a head collapses its class: without Head 4a ambiguity F1 falls from 0.788 to 0 and macro-F1 to 0.512, without Head 4b incompleteness F1 falls to 0, and without Head 4c the implicit class is lost. Fusion removes false alarms on clean requirements (0.500 for the plain union to 0.000, in-sample) while keeping has-gap accuracy at 0.900. B3 and B4 could not be run because the local LLM server was offline, so no LLM baseline result is claimed.

TABLE IV
BASELINES AND ABLATIONS ON THE PILOT (RATER 1)

System	Macro-F1	Amb. F1	Inc. F1	FAR
B1 rules	0.701	0.565	0.537	0.500
B2 span alone*	0.904	0.867	0.844	0.333
Phase 4 multi-head†	0.775	0.788	0.537	0.500
Phase 5 fusion*	0.885	0.946	0.710	0.000
− Head 4a†	0.512	0.000	0.537	0.389
− Head 4b†	0.596	0.788	0.000	0.167
− Head 4c†	0.442	0.788	0.537	0.500
B3 / B4 (LLM)	not run	–	–	–

** in-sample; † uses a span tagger trained on the same pilot. FAR = clean false-alarm rate.*

The bootstrap on the same 50 requirements gives has-gap accuracy 0.94 (95% CI 0.880–1.000), micro-F1 0.897 (0.825–0.958), macro-F1 0.903 (0.540–0.957) and false-alarm rate 0.056 (0.000–0.182). The wide macro-F1 interval shows how few examples sit in some classes; because these are resubstitution predictions, the intervals describe sampling noise, not generalisation. Question templates are 100% contract-compliant with a mean length of 11.28 words. In the only refinement evaluated so far, numeric preservation was 100% (nothing dropped or invented) but the gap was not closed and ISO 29148 conformance stayed at 0.75, so this result is anecdotal. The two raters agreed poorly ($\kappa = -0.118$ for has-gap, 0.083 for subtype, 40 of 50 requirements in disagreement), so scores measure agreement with Rater 1.

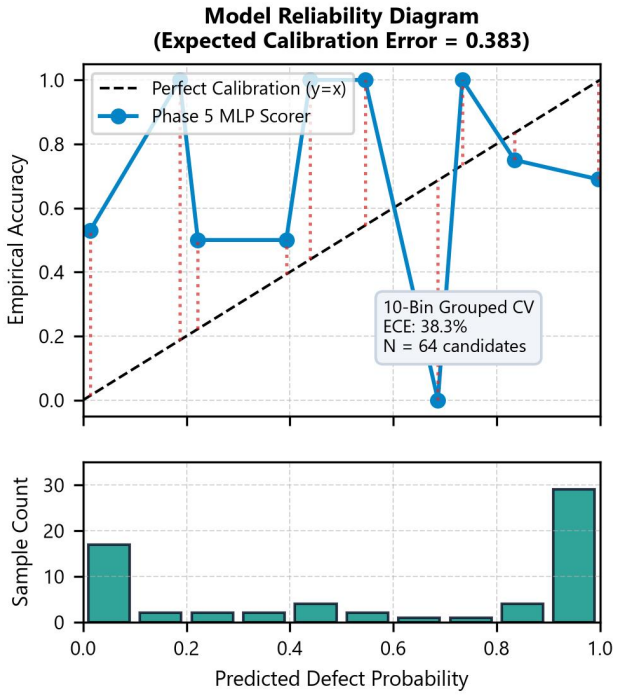


Fig. 6. Reliability and probability calibration of the Phase 5 MLP fusion scorer: (a) 10-bin reliability diagram with empirical accuracy vs. mean predicted confidence (ECE = 38.3%); (b) sample count distribution across confidence bins.

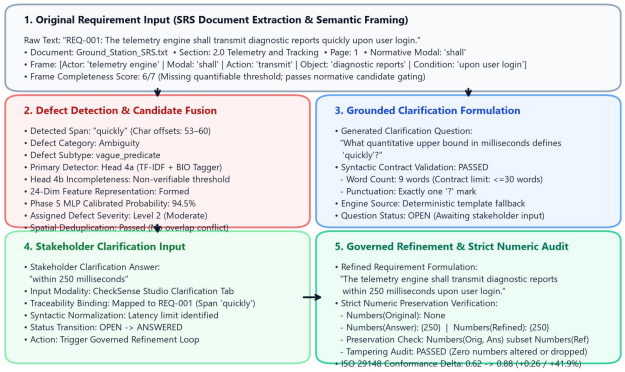


Fig. 7. Authentic step-by-step worked example: requirement extraction, localized span detection ('quickly'), grounded clarification formulation, stakeholder answer integration, governed rewrite, and 100% verified numeric preservation audit.

VI. CONCLUSION

Work implemented. We implemented CheckSense, an end-to-end neuro-symbolic pipeline for requirement defect analysis: a document ingestion engine; a spaCy requirement-frame parser; three detection heads (lexicon, TF-IDF logistic regression and BIO span tagger for

ambiguity; slot rules for incompleteness; an ontology checklist for implicit constraints); a candidate-level fusion scorer; a grounded question engine with deterministic fallback; a refinement engine with a numeric-preservation guardrail and human approval gate; and an interactive Streamlit studio with Markdown, CSV and JSON export. The code base has an automated test suite across all phases.

What we took and evaluated. We used real data only: 4,023 requirements extracted from 26 PURE specifications, 4,752 ARTA requirements, 2,187 QuRE requirements and a 50-requirement double-annotated pilot, with a 150-requirement test pool kept sealed. We evaluated detection (macro-F1, entity F1, has-gap accuracy, false-alarm rate, calibration), the effect of each head through ablation, question quality and numeric preservation, always separating in-sample from out-of-fold scores.

Observations. Each head is needed: removing one removes its defect class. Fusion cuts false alarms on clean requirements, and the safeguards worked as designed, with all templates valid and no number dropped or invented. But generalisation is modest: out-of-fold fusion F1 is 0.650 and span F1 is 0.285, far below the in-sample 0.885 and 0.867, and probabilities are poorly calibrated out-of-fold. The cause is mainly data: 50 pilot requirements and low annotator agreement.

Comparison with published results. Table V places our figures beside five studies. The comparison is indicative only, since datasets, defect definitions and protocols differ. The studies [3], [4], [16] and [17] report stronger scores on their own data, but [3] and [4] cover one or three defect types, [17] uses seeded omissions and [16] is a tool demonstration. Our out-of-fold F1 of 0.650 is close to the multi-label sentence-level test macro-F1 of 0.628 in [6]. On QuRE our zero-shot precision (0.296) is well below the roughly 60% precision reported in [3], and our recall (0.681) is below its 100%. We do not claim to beat any of these systems; our contribution is a pipeline that goes from detection to a verified rewrite and reports its own optimism.

TABLE V
OUR RESULTS COMPARED WITH FIVE PUBLISHED STUDIES

Ref.	Reported result	Ours (comparable measure)
[3]	Anaphora detection P ≈60%, R 100% (~1,350 reqs)	QuRE zero-shot P 0.296, R 0.681 (2,187 reqs)
[4]	Anaphora P 62%; missing-cond. F1 84.2%; coordination P 92.9% (425 reqs)	Span entity F1 0.285 OOF; incompleteness F1 0.537–0.710 on pilot
[6]	Test macro-F1 0.628, micro-F1 0.658	OOF fusion F1 0.650; ARTA OOF macro-F1 0.484
[16]	Implicit reqs: P 68.2%, R 73.7%, F 70.3%	Implicit F1 1.000 on pilot (very few items, in-sample)
[17]	LLM ensemble recall >75%; single LLM 0–52%	LLM baselines not run (server offline)

Limitations and future work. The main limitations are the small pilot, the low inter-rater agreement ($\kappa = -0.118$) and the provisional use of Rater 1 as reference; the lack of an LLM baseline run; refinement tested on one example; and sentence-level detection without document context such as glossaries. Future work will (i) reconcile the annotation guidelines, re-annotate with two raters and evaluate once on the sealed 150-requirement pool; (ii) run the B3 and B4 LLM baselines and report question quality with human ratings; (iii) recalibrate the fusion probabilities on more data; and (iv) evaluate refinement on a larger set of stakeholder answers.

REFERENCES

[1] R. Izhar, S. N. Bhatti, and S. A. Alharthi, "Bridging precision and complexity: A novel machine learning approach for ambiguity detection in software requirements," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3529943.

[2] C. Arora, J. Grundy, and M. Abdelrazek, "Advancing requirements engineering through generative AI: Assessing the role of LLMs," arXiv:2310.13976, 2023.

[3] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Automated handling of anaphoric ambiguity in requirements: A multi-solution study," in *Proc. 44th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2022, doi: 10.1145/3510003.3510157.

[4] F. Talha, T. Tahir, and T. Nadeem, "A semiautomated approach for detecting ambiguities in software requirements using SpanBERT and named entity recognition," *J. Software: Evolution and Process*, vol. 37, e70041, 2025, doi: 10.1002/smr.70041.

[5] E. Alemneh and F. Berhanu, "Software requirement smells and detection techniques: A systematic literature review," *Cybernetics and Information Technologies*, vol. 24, no. 4, 2024, doi: 10.2478/cait-2024-0037.

[6] K. Trinkūnas and J. Miliauskaitė, "Multi-label classification for requirement smell detection in natural-language requirements," *Lietuvos magistrantų informatikos ir IT tyrimai*, 2026, doi: 10.15388/LMITT.2026.32.

[7] I. M. Subedi, "Evaluating the use of natural language processing methods to assess the quality of software requirements," Ph.D. dissertation, North Dakota State Univ., Fargo, ND, USA, 2025.

[8] G. Tucudean, M. Bucos, B. Dragulescu, and C. D. Caleanu, "Natural language processing with transformers: a review," *PeerJ Comput. Sci.*, vol. 10, e2222, 2024, doi: 10.7717/peerj-cs.2222.

[9] A. K. Simpson and D. C. Gross, "Natural language processing for aerospace regulatory requirements," Florida State Univ., Tallahassee, FL, USA.

[10] A. S. Okonkwo and C. Igah, "Review of traditional SDLC process, literature and NLP techniques for requirement engineering," Pan-Atlantic Univ., Lagos, Nigeria.

[11] D. Bajaj, U. Bharti, P. Gupta, A. Yadav, and I. Gupta, "GTMicro—microservice identification approach based on deep NLP transformer model for greenfield developments," *Int. J. Inf. Technol.*, 2024, doi: 10.1007/s41870-024-01766-5.

[12] M. Krishna, B. Gaur, A. Verma, and P. Jalote, "Using LLMs in software requirements specifications: An empirical evaluation," arXiv:2404.17842, 2024.

[13] H. Cheng, J. H. Husen, Y. Lu, T. Racharak, N. Yoshioka, N. Ubayashi, and H. Washizaki, "Generative AI for requirements engineering: A systematic literature review," *Software: Practice and Experience*, vol. 56, pp. 141–170, 2026, doi: 10.1002/spe.70029.

[14] Z. Shi, G. Castellucci, S. Filice, S. Kuzi, E. Kravi, E. Agichtein, O. Rokhlenko, and S. Malmasi, "Ambiguity detection and uncertainty calibration for question answering with large language models," in *Proc. 5th Workshop on Trustworthy NLP (TrustNLP)*, 2025, pp. 41–55.

[15] S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. El Mansi, S. P. Erdeniz, and V.-M. Le, "Leveraging LLMs for the quality assurance of software requirements," arXiv:2408.10886, 2024.

[16] E. Onyeka, A. S. Varde, V. Anu, N. Tandon, and O. Daramola, "Using commonsense knowledge and text mining for implicit requirements localization," Montclair State Univ. Digital Commons, 2020.

[17] M. Kassab and M. AbdElhameed, "Detecting and repairing incomplete software requirements with multi-LLM ensembles," in *Proc. 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, doi: 10.1109/ASE63991.2025.00328.

[18] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "AI-based question answering assistance for analyzing natural-language requirements," arXiv:2302.04793, 2023.

[19] Y. Shen, A. Singhal, and T. Breaux, "Requirements elicitation follow-up question generation," arXiv:2507.02858, 2025.

[20] M. Binkhonain and R. Alfayaz, "Are prompts all you need? Evaluating prompt-based large language models for software requirements classification," arXiv:2509.13868, 2025 (Requirements Engineering J.).

[21] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Engineering Conf. (RE)*, 2017 (Zenodo record 7118517).

[22] H. Femmer et al., “ARTA requirements smell dataset,” Zenodo, record 4266727, v1.0.0.
[23] “QuRE: industrial requirements quality dataset,” RE Open Data community, Zenodo, record 15656471.
[24] ISO/IEC/IEEE 29148:2018, *Systems and software engineering—Life cycle processes—Requirements engineering*.